

MEMORIA:

Proyecto de clasificación de cartas

Nuria Urrecho Torres

nurrtor@etsinf.upv.es

VCO

1.Desarrollo

1.1. Extracción de características

Para esta etapa, se ha seguido el guion de la práctica 6, empleando además los códigos proporcionados por el profesor: `clases_cartas.py` y `segmentar_imagen.py`.

En primer lugar, se define la clase **Card**, que es la estructura de datos principal del programa. Cada objeto de esta clase representa una carta detectada en la imagen. La clase incluye constantes que definen los posibles palos y figuras de una carta de póker, así como un conjunto de atributos que almacenan toda la información relevante asociada a una carta: un identificador único, el bounding box original, el ángulo de rotación aplicado, la imagen de la carta en escala de grises, la imagen en color, una máscara binaria de la carta y una lista de motivos internos. Esta estructura permite encapsular toda la información necesaria para el análisis posterior y facilita el manejo de múltiples cartas de forma simultánea.

También se define la clase **Motif**, pensada para representar cada símbolo o número presente dentro de una carta.

A continuación, se implementa la función **label2rgb**, cuya finalidad es convertir una imagen de etiquetas obtenida mediante componentes conectadas en una imagen en color, en la cual a cada etiqueta numérica se asigna a un tono distinto.

Una de las funciones clave del programa es **segmentar_objetos_carta**. Esta función recibe como argumento un objeto de la clase Card y se encarga de detectar y analizar los motivos presentes en su interior. En primer lugar, la función comprueba que la carta contiene una imagen válida en escala de grises, evitando así errores. A continuación, se aplica una umbralización binaria invertida sobre la imagen de la carta, de forma que los motivos oscuros aparecen como objetos blancos sobre un fondo negro. Esta representación permite aplicar de manera directa el análisis de componentes conectadas.

Mediante la función **cv2.connectedComponentsWithStats**, se identifican todos los objetos presentes en la imagen binaria, obteniendo para cada uno

su área, su bounding box y su centroide. Para eliminar ruido y detalles irrelevantes, se aplica un filtrado por área mínima, descartando aquellos objetos demasiado pequeños. Para cada motivo válido se genera una máscara binaria, se extrae su contorno externo y se dibuja sobre una imagen de visualización. Además, se calculan diversas características geométricas como el área real del contorno, el perímetro, el centroide y los momentos Hu (que son especialmente útiles para el reconocimiento de formas), y también se calcula el color medio del motivo a partir de la imagen en color de la carta, lo que permite diferenciar entre palos rojos y negros.

Todas estas características se almacenan en un vector NumPy asociado a cada motivo y, finalmente, la función genera una imagen con los motivos dibujados y la almacena en el objeto Card.

Tras la definición de funciones y clases, el programa solicita al usuario la selección de una carpeta que contenga las imágenes a procesar. Se inicializa una lista vacía denominada Cards, que almacenará todos los objetos Card creados durante la ejecución, y un contador que asigna un identificador único a cada carta detectada.

El núcleo del programa se encuentra en el bucle principal que recorre todas las imágenes de la carpeta seleccionada. Cada imagen se lee en color y se convierte a escala de grises para después aplicar una umbralización global invertida y obtener su imagen complementaria, sobre la cual se realiza el análisis de componentes conectadas. Esto permite que las cartas aparezcan en blanco, mientras que los motivos internos no se ven.

Cada componente conectada se analiza individualmente y se filtra por área (aquellas componentes cuya área supera un umbral elevado se consideran cartas completas). Para cada carta detectada se genera una máscara binaria, se obtienen sus contornos y se calcula su bounding box. A partir de este bounding box se recorta la carta tanto en la imagen en escala de grises como en la imagen original en color.

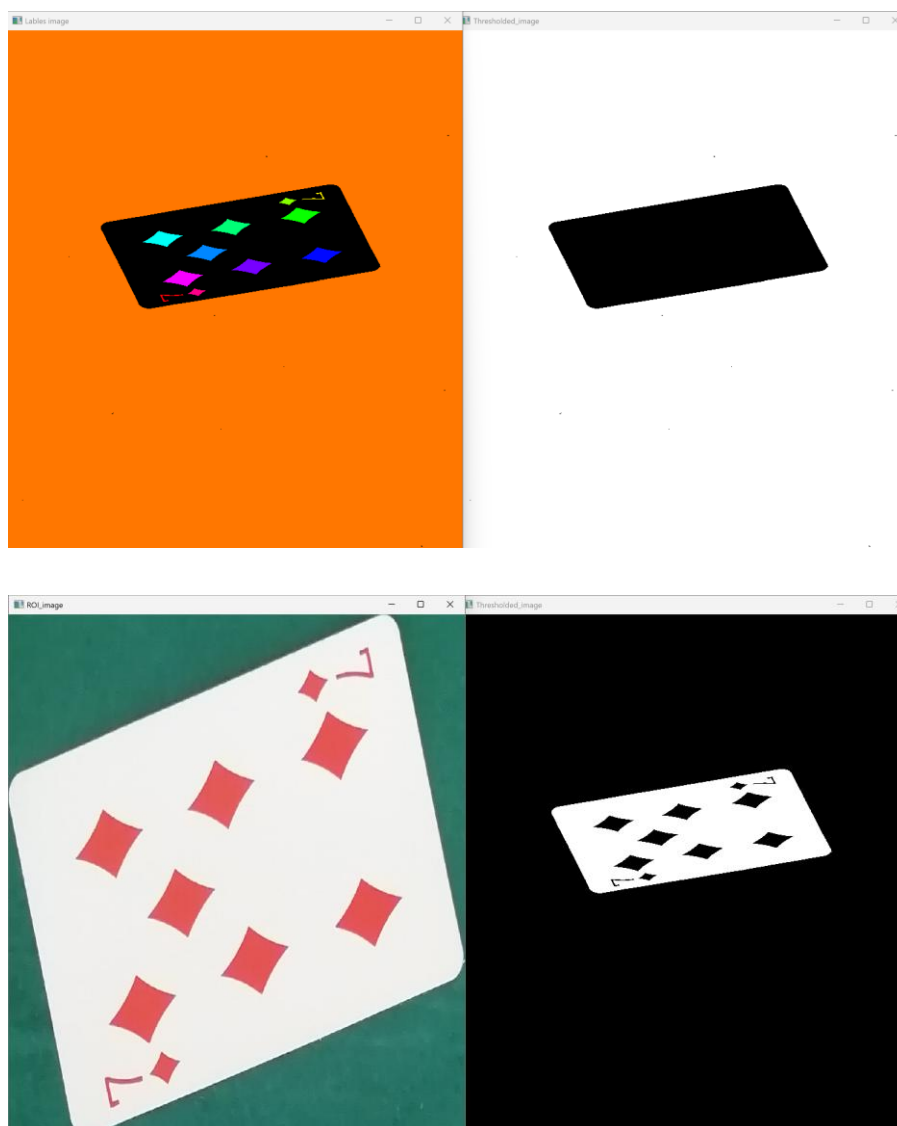
Dado que las cartas pueden aparecer giradas en la imagen, se calcula el rectángulo mínimo rotado que encierra el contorno de la carta mediante la función `cv2.minAreaRect`. A partir de este rectángulo se obtiene el ángulo de rotación necesario para alinear la carta, el cual se ajusta para garantizar una orientación vertical y se construye una matriz de rotación que se aplica tanto a la imagen de la carta como a su máscara binaria.

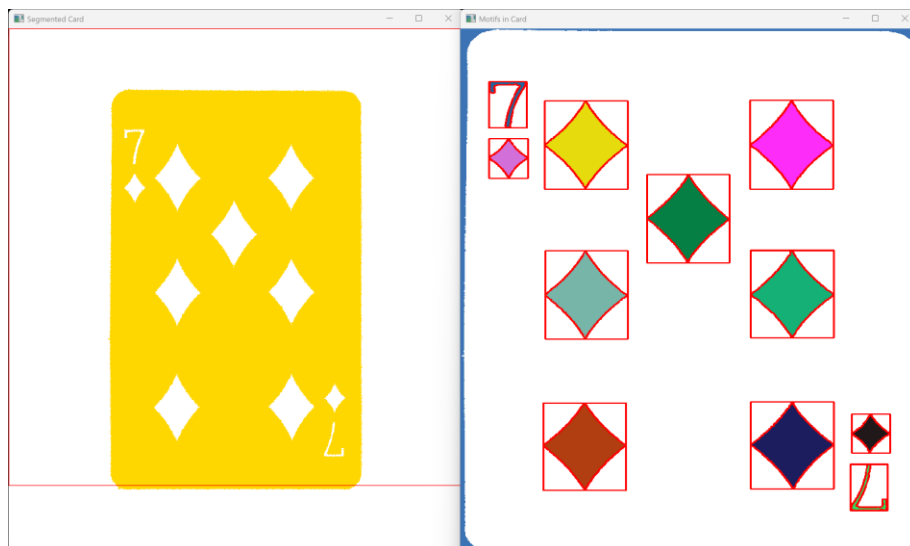
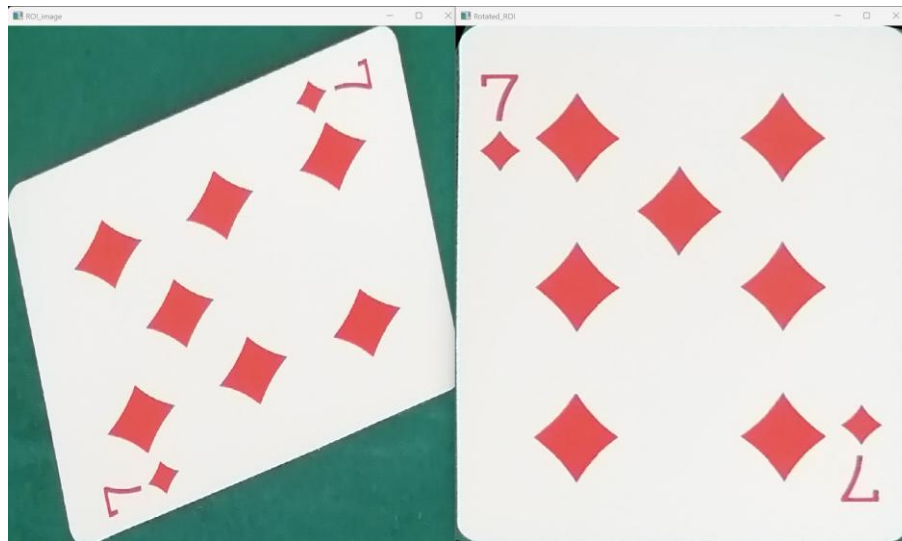
Una vez corregida la orientación, se genera una imagen segmentada de la carta, se dibujan sobre la imagen original tanto el bounding box como el rectángulo mínimo rotado, y se crea un objeto de la clase Card donde se almacenan todos los datos obtenidos.

Tras la creación de cada carta, se llama a la función ***segmentar_objetos_carta***, que analiza el contenido interno de la carta y extrae los motivos.

Finalmente, una vez procesadas todas las imágenes, el programa guarda la lista completa de cartas en un archivo `cartas.npz`.

A continuación, se muestran algunas capturas de pantalla del proceso hasta llegar al resultado final (última captura):





1.2. Clasificadores estadísticos

Segmentar_cartas.py

Partimos del código proporcionado en Poliformat `segmentar_cartas.py`.

En primer lugar, se deben crear las carpetas `train` y `test` dentro del directorio de las imágenes. Entonces debemos introducir en `train` todas las imágenes en las que solo haya una carta y en `test` en las que haya más de una.

El funcionamiento general del programa consiste en recorrer un conjunto de imágenes, detectar las cartas presentes en cada una de ellas, segmentar los motivos internos (símbolos y figuras), extraer sus características y almacenar toda esta información estructurada en archivos `.npz` diferenciando conjuntos de entrenamiento y de test.

La principal aportación añadida respecto al código base se es la función **`segmentar_objetos_carta`**. En esta función, una vez obtenidos los contornos de los motivos y análisis de componentes conectadas, se realiza un cálculo de características.

Se han añadido features como la circularidad, rectangularidad, elongación, densidad, solidez, etc. Esta gran cantidad de medidas se añadieron para mejorar el correcto análisis de los datos, facilitando su uso posterior en clasificadores como k-NN y mejorando potencialmente la tasa de acierto.

Otro elemento relevante es la función `VISUALIZAR`. Esta nos permite mostrar o no cada motivo detectado individualmente, lo que resulta útil para comprobar la correcta detección de los motivos. Sin embargo, para segmentar las cartas automáticamente lo pondremos en `false` para acelerar el proceso.

Finalmente, se realiza el almacenamiento de los datos en los `.npz`. Una vez procesadas todas las imágenes, el script guarda la información obtenida en archivos `trainCards.npz` y `testCards.npz`.

Etiquetar_cartas_OK.py

Partimos del código proporcionado en Poliformat `etiquetar_cartas_OK.py`

El programa comienza cargando uno de los archivos `.npz`. Cada carta incluye su imagen, la lista de motivos detectados y los vectores de características asociados a dichos motivos. A partir de este momento, el script trabaja directamente sobre estas estructuras de datos.

Es entonces cuando el usuario clasifica manualmente cada motivo a través de la interfaz gráfica. Por esto, se debe ejecutar el programa dos veces, una para cada archivo `trainCards.npz` y `testCards.npz`.

Al finalizar el etiquetado, el script guarda automáticamente los cambios en el archivo `.npz` correspondiente.

Preprocesar_npz.py

Este archivo ha sido implementado por la necesidad de mejorar los datos que posteriormente serán procesados por el clasificador KNN, ya que en un principio los resultados que obteníamos eran demasiado bajos. Esto lo conseguiremos actuando exclusivamente sobre los datos (normalizándolos y tratando los valores atípicos).

En primer lugar, el código carga los objetos `Card` desde los archivos `.npz` y extrae los vectores de características, comprobando que todos tengan alguna característica y que los vectores sean de la misma dimensión.

A continuación, se realiza un recorte de outliers para evitar los valores extremos usando el conjunto de entrenamiento. Además, también se realiza un escalado que centra cada característica en torno a su mediana y la normaliza. Cabe destacar que este escalado se calcula con el conjunto de entrenamiento y, ese mismo, se aplica al conjunto de test para asegurar que el rendimiento medido del clasificador es realista.

Finalmente, se sobrescriben los vectores de características de cada motivo y se guardan en sus respectivos archivos `.npz`.

Modelo_knn.py

Partimos del código proporcionado en Poliformat `modelo_kNN.py`

En primer lugar, se carga el conjunto de entrenamiento y se construyen dos listas: *samples* (vectores de características) y *responses* (etiquetas reales de cada motivo). Entonces, se obtiene (para cada motivo etiquetado) su vector de características (*mot.features*). Este se convierte en un vector numérico y se almacena junto a su etiqueta real. Después, se crea el clasificador KNN y se entrena con la matriz de características y las etiquetas reales.

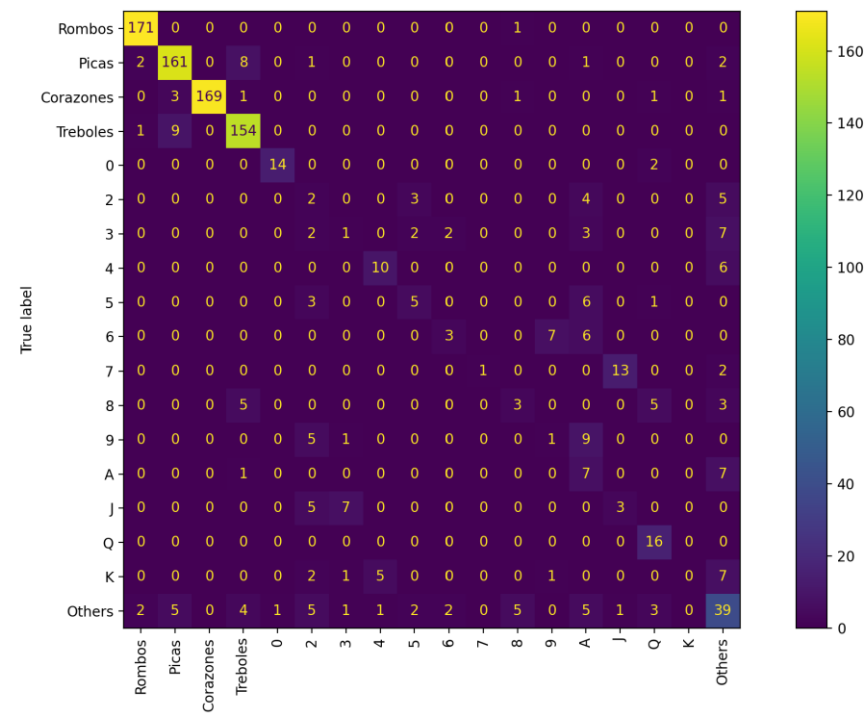
En segundo lugar, se carga el conjunto de test y se repite el mismo proceso de extracción de características. Es entonces cuando se realiza la predicción con

k=1 (por ser la que mejores resultados nos ofrece) y se obtiene la clase predicha, los vecinos más cercanos y la distancia asociada.

Finalmente, se calcula la tasa de aciertos, el Classification report, la matriz de confusión y el MCC. Estos son los resultados:

MCC: 0.7526751338495662

Tasa aciertos: 0.7859358841778697				
Classification report:				
	precision	recall	f1-score	support
Rombos	0.97	0.99	0.98	172
Picas	0.90	0.92	0.91	175
Corazones	1.00	0.96	0.98	176
Treboles	0.89	0.94	0.91	164
0	0.93	0.88	0.90	16
2	0.08	0.14	0.10	14
3	0.09	0.06	0.07	17
4	0.62	0.62	0.62	16
5	0.42	0.33	0.37	15
6	0.43	0.19	0.26	16
7	1.00	0.06	0.12	16
8	0.30	0.19	0.23	16
9	0.11	0.06	0.08	16
A	0.17	0.47	0.25	15
J	0.18	0.20	0.19	15
Q	0.57	1.00	0.73	16
K	0.00	0.00	0.00	16
Others	0.49	0.51	0.50	76
accuracy			0.79	967
macro avg	0.51	0.47	0.46	967
weighted avg	0.79	0.79	0.78	967



1.3. Redes neuronales

Obtener_imagenes_motivos.py

En primer lugar, se carga el archivo .npz y se define un directorio base donde se guardarán las imágenes de los motivos. Dentro del mismo, se crea automáticamente una subcarpeta por cada clase usando el diccionario LABEL_MAP.

Seguidamente, se realiza un recorte de cada motivo en función del contorno y el bounding box. Cada motivo será almacenado en su carpeta correspondiente en formato png y, además, se generarán tres versiones rotadas para aumentar el número de muestras por clase y mejorando la robustez del modelo.

El programa se ejecutará dos veces, una para el conjunto de entrenamiento y otra para el conjunto de test.

myCNN.py

En primer lugar, se importan las librerías necesarias. En nuestro caso, hemos añadido de las siguientes librerías:

```
from tensorflow.keras.applications import MobileNetV2
from sklearn.utils.class_weight import compute_class_weight
```

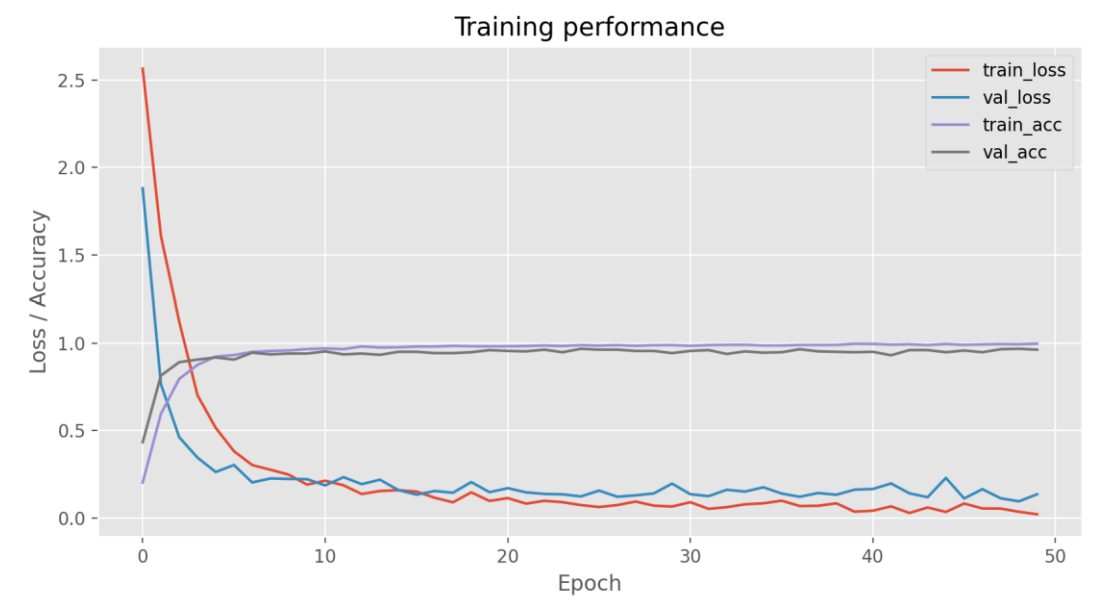
- **MobileNetV2:** en vez de entrenar una red convolucional desde cero, se emplea este modelo convolucional preentrenado que ha aprendido previamente características visuales que resultan útiles para la solución del problema.
- **Compute_class_weight:** para balancear el número de muestras por clase.

Seguidamente, el código carga las imágenes del conjunto de entrenamiento y filtra las clases con un mínimo de 5 muestras. Para generar los conjuntos de entrenamiento y validación se usa ImageDataGenerator empleando la técnica antes mencionada de data augmentation. Además, se emplea validation_split=0.2 para separar un 80% y un 20% entre entrenamiento y validación respectivamente.

Posteriormente, observamos que el conjunto de datos presentaba un número desigual de muestras por clase. Para solucionarlo, empleamos la función *class_weight*. También se aplica una técnica llamada Transfer Learning empleando MobileNetV2, para mejorar el rendimiento de la red.

Entonces, se construye el modelo CNN empleando varias capas Conv2D, MaxPool2D, Flatten, Dense, Dropout y una capa final Dense con softmax, cuyo número de neuronas coincide con el número de clases.

Finalmente, se compila el modelo con SGD y se entrena el modelo con *model.fit* con las clases balanceadas. Los resultados obtenidos se visualizarán mediante una gráfica de pérdida y precisión y se guardarán en el archivo myCNN.h5.



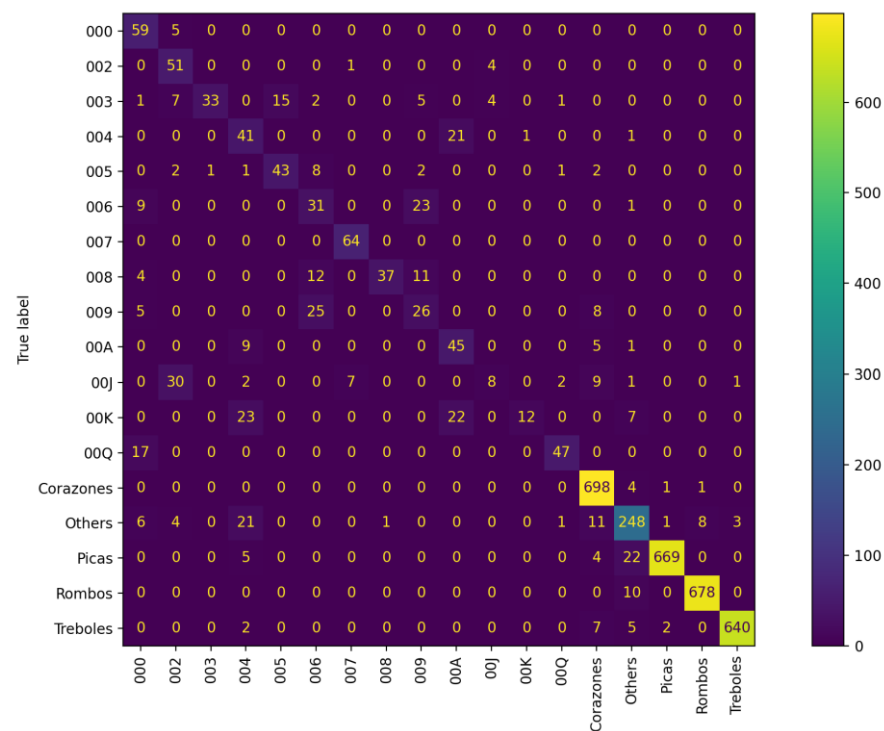
De la gráfica podemos observar que el modelo está aprendiendo correctamente, principalmente porque alcanza buen rendimiento en pocas épocas y luego solo refina, además de que las curvas de *train_loss* y *val_loss* descienden rápidamente y luego se mantienen bajas. Por último, las curvas de train y val son muy similares tanto en loss como en accuracy, lo que confirma que no hay sobreajuste y el modelo aprende correctamente de los patrones.

Evaluación_CNN.py

Este programa leerá el archivo *myCNN.h5* y analizará parámetros como la precisión, la matriz de confusión, el MCC, etc. Para realizarlo, se ha partido del código proporcionado en el apartado 6 de la práctica. Los resultados se muestran a continuación

MCC: 0.8695

Predictions shape: (3868, 18)				
	precision	recall	f1-score	support
000	0.58	0.92	0.72	64
002	0.52	0.91	0.66	56
003	0.97	0.49	0.65	68
004	0.39	0.64	0.49	64
005	0.74	0.72	0.73	60
006	0.40	0.48	0.44	64
007	0.89	1.00	0.94	64
008	0.97	0.58	0.73	64
009	0.39	0.41	0.40	64
00A	0.51	0.75	0.61	60
00J	0.50	0.13	0.21	60
00K	0.92	0.19	0.31	64
00Q	0.90	0.73	0.81	64
Corazones	0.94	0.99	0.96	704
Others	0.83	0.82	0.82	304
Picas	0.99	0.96	0.97	700
Rombos	0.99	0.99	0.99	688
Treboles	0.99	0.98	0.98	656
accuracy			0.89	3868
macro avg	0.75	0.70	0.69	3868
weighted avg	0.90	0.89	0.88	3868



2. Conclusión

Tras todo el proceso y obtener el informe final del modelo h5, podemos comparar ambos modelos y vemos que claramente nuestro modelo CNN es más efectivo que el Knn.

Sabiendo esto, podemos sacar diversas conclusiones con los resultados obtenidos.

En primer lugar, la exactitud (*accuracy*) es de un 89%, lo que indica que el sistema clasifica correctamente la gran mayoría de las muestras. No obstante, la exactitud por sí sola no da una visión completa del comportamiento del modelo.

Es por esto que se deben analizar también las otras métricas: precisión, *recall* y *F1-Score*, así como promedios macro y ponderados. De esta forma, el análisis de *F1-Score macro* (0.69) muestra que el rendimiento del modelo no es igual entre todas las clases, así como valores de *recall* bajos. Esto indica que el modelo no detecta correctamente todas las muestras reales de las cartas, probablemente por su similitud.

Podemos verlo reflejado, por ejemplo, en la clase "008". Esta presenta una precisión de 0.97 pero un *recall* de 0.58 lo que implica que, aunque el modelo acierta el 97% de las veces, solo consigue reconocer el 58% de las veces que aparece realmente.

Por otro lado, el *Weighted average* (promedio ponderado) muestra valores elevados de precisión, *recall* y *F1-Score* (alrededor de 0.89), lo que confirma que el sistema funciona especialmente bien en las clases con mayor número de muestras. Esto se corresponde con el comportamiento con los palos, donde los valores de *precision* y *recall* son prácticamente perfectos. Todo esto significa que las características de los palos son fácilmente reconocibles por el modelo.

Sin embargo, esta diferencia de rendimiento entre clases justifica que el *MCC* sea la métrica principal para evaluar el sistema. En este caso, el *MCC* obtenido es de 0.8695, lo que confirma la alta correlación entre las predicciones y las clases reales.

Por todo esto, los resultados obtenidos en la evaluación del modelo de reconocimiento de cartas nos permiten concluir que el modelo tiene un rendimiento global elevado, aunque con diferencias significativas entre clases.

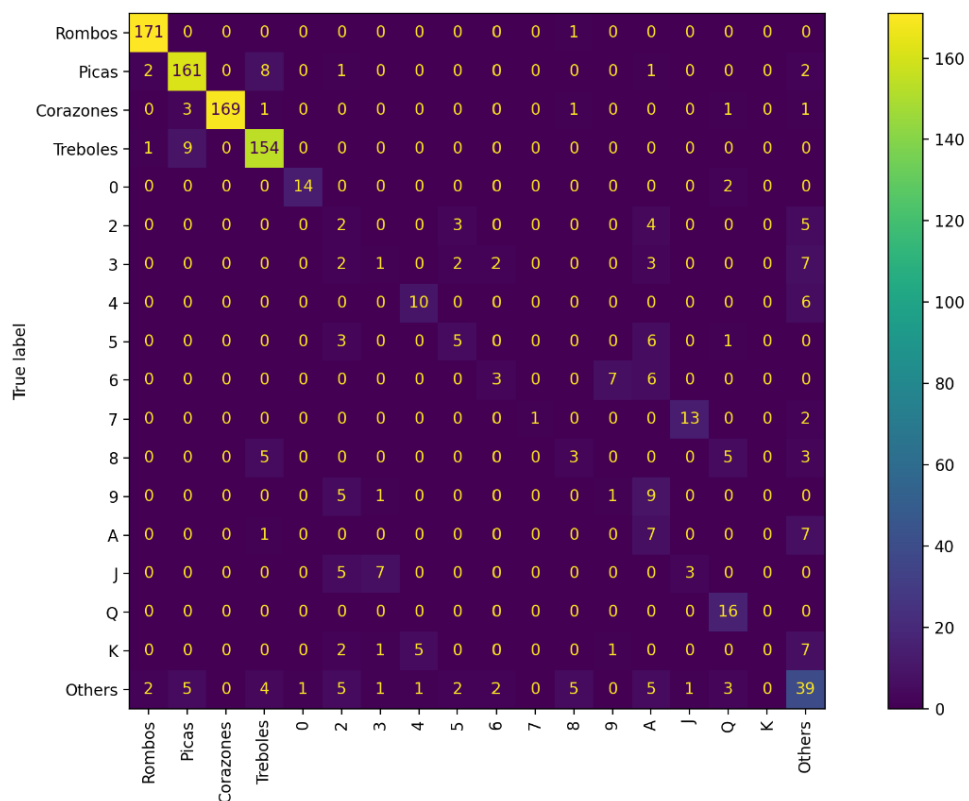
A continuación, en el siguiente apartado, se muestran las tablas con los resultados de ambos modelos para poder compararlos correctamente.

3. Tablas

A continuación, se muestran los resultados en tabla del modelo Knn:

	precision	recall	F1-score	support
Rombos	0.97	0.99	0.98	172
Picas	0.90	0.92	0.91	175
Corazones	1.00	0.96	0.98	176
Tréboles	0.89	0.94	0.91	164
0	0.93	0.88	0.90	16
2	0.08	0.14	0.10	14
3	0.09	0.06	0.07	17
4	0.62	0.62	0.62	16
5	0.42	0.33	0.37	15
6	0.43	0.19	0.26	16
7	1.00	0.06	0.12	16
8	0.30	0.19	0.23	16
9	0.11	0.06	0.08	16
A	0.17	0.47	0.25	15
J	0.18	0.20	0.19	15
Q	0.57	1.00	0.73	16
K	0.00	0.00	0.00	16
Others	0.49	0.51	0.50	76
Accuracy			0.79	967
Macro avg	0.51	0.47	0.46	967
Weighted avg	0.79	0.79	0.78	967

MCC: 0.7527



A continuación, se muestran los resultados en tabla de la red CNN:

	precision	recall	F1-score	support
Rombos	0.99	0.99	0.99	688
Picas	0.99	0.96	0.97	700
Corazones	0.94	0.99	0.96	704
Tréboles	0.99	0.98	0.98	656
0	0.58	0.92	0.72	64
2	0.52	0.91	0.66	56
3	0.97	0.49	0.65	68
4	0.39	0.64	0.49	64
5	0.74	0.72	0.73	60
6	0.40	0.48	0.44	64
7	0.89	1.00	0.94	64
8	0.97	0.58	0.73	64
9	0.39	0.41	0.40	64
A	0.51	0.75	0.61	60
J	0.50	0.13	0.21	60
Q	0.92	0.19	0.31	64
K	0.90	0.73	0.81	64
Others	0.83	0.82	0.82	304
Accuracy			0.89	3868
Macro avg	0.75	0.70	0.69	3868
Weighted avg	0.90	0.89	0.88	3868

MCC: 0.8695

